



- ♦ Spring Data Jpa not only provides implementation for commonly used methods but also provides a way to add custom methods.
- ♦ Spring Data only provides the implementation for custom methods.
- ♦ Spring Data defines Domain-Specific Language (DSL) where persistence details are expressed in method signature.

Ex:

```
interface CustomerDAO extends JpaRepository<Customer, Integer>{  
    List<Customer> findCustomerByCname(String name);  
    List<Customer> getCustomerByEmail(String email);  
  
    List<Customer> readByPhone(long phone);  
  
    List<Customer> getCustomerByEmailAndPhone(String email,long phone);  
    List<Customer> findCustomerByEmailOrPhone(String email,long phone);  
}
```

- ♦ Repository methods are combination of the following 4 things.

1. **Verb,**
2. **Optional Subject,**
3. **word By,**
4. **Predicate.**

Ex1:

```
findCustomerByEmail(String email)  
find is verb  
Customer is subject  
By is word  
Email is predicate
```

Ex2:

```
findByEmail(String email)  
find is verb  
the subject isn't specified and is implied to be a Customer.  
By is word  
Email is predicate
```

Ex3:

```
readCustomerByEmailOrPhone(String email, long phone)  
read is verb  
Customer is subject  
By is word  
EmailOrPhone is predicate
```



- ♦ Spring Data allows four verbs in the method name:
 - **get**
 - **read**
 - **find**
 - **count.**
- ♦ **get**, **read**, and **find** verbs are synonymous; all three query for data and return object(s).

```
List<Customer> getCustomerByEmail(String email);  
List<Customer> findCustomerByEmail(String email);  
List<Customer> readCustomerByEmail(String email);
```

- ♦ The **count** verb returns a count of matching objects, rather than the objects themselves.

```
long countCustomerByEmail(String email);
```

- ♦ The predicate is the most interesting and important part of the method name.
- ♦ Predicate contains Entity Class Properties and Operators
- ♦ We can use any of the following operators in Predicate

- 1) **IsAfter, After, IsGreaterThan, GreaterThan**
- 2) **IsGreaterThanEqual, GreaterThanEqual**
- 3) **IsBefore, Before, IsLessThan, LessThan**
- 4) **IsLessThanEqual, LessThanEqual**
- 5) **IsBetween, Between**
- 6) **IsNull, Null**
- 7) **IsNotNull, NotNull**
- 8) **IsIn, In**
- 9) **IsNotIn, NotIn**
- 10) **IsStartingWith, StartingWith, StartsWith**
- 11) **IsEndingWith, EndingWith, EndsWith**
- 12) **IsContaining, Containing, Contains**
- 13) **IsLike, Like**
- 14) **IsNotLike, NotLike**
- 15) **IsTrue, True**
- 16) **IsFalse, False**
- 17) **Is, Equals**
- 18) **IsNot, Not**



- ♦ We can sort the results by adding OrderBy at the end of the method name.
- ♦ Sort the results in ascending order by the cname.

```
List<Customer> readByCityAscCname(String city);
```

- ♦ Sort the results in descending order by the cname.

```
List<Customer> readByCityDescCname(String city);
```

- ♦ Spring Data DSL has its limitations, and sometimes it isn't convenient or even possible to express the desired query in a method name.
- ♦ In this case, Spring Data provides @Query annotation to write query explicitly.
- ♦ if we followed the naming convention, the method name would be incredibly long. In such situation, @Query can be useful.
- ♦ The @Query annotation is handy for adding custom query methods to a Spring Data JPA-enabled interface.

demo3-data-jpa: Files required

1. MyBootApplication.java	Updated in Demo3
2. Customer.java	Updated in Demo3
3. CustomerDAO.java	Updated in Demo3
4. application.properties	Same as Demo2
5. pom.xml	Same as Demo2
6. mycustomers table	Updated in Demo3

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {
```



```
@Autowired
CustomerDAO custDAO;

@Override
public void run(String... args) throws Exception {
/*

//Insert 9 Records
Customer cust1 = new Customer("Sri", "Sri@jlc", 111, "Blore",5000);
Customer cust2 = new Customer("sd", "sd@jlc", 222, "Blore",25000);
Customer cust3 = new Customer("ds", "ds@jlc", 333, "Hyd",15000);
Customer cust4 = new Customer("hello", "hello@jlc", 12345, "Blore",15000);
Customer cust5 = new Customer("Sri", "Sri@jlc", 12345, "Delhi",9000);
Customer cust6 = new Customer("sd", "sd@jlc", 12345, "Blore",18000);
Customer cust7 = new Customer("ds", "ds@jlc", 333, "Delhi",17000);
Customer cust8 = new Customer("Sri", "Sri@jlc", 12345, "Hyd",23000);
Customer cust9 = new Customer("Hai", "hai@cc", 12345, "Hyd",23000);

List<Customer> custList =
Arrays.asList(cust1,cust2,cust3,cust4,cust5,cust6,cust7,cust8,cust9);

custDAO.saveAll(custList);
*/

List<Customer> mylist=null;

//1. getCustomersByCname
mylist=custDAO.getCustomersByCname("Sri");
mylist.forEach(System.out::println);

//2. getCustomersByEmail
mylist=custDAO.getCustomersByEmail("hello@jlc");
mylist.forEach(System.out::println);

//3. readCustomersByPhone
mylist=custDAO.readCustomersByPhone(12345);
mylist.forEach(System.out::println);

//4. findCustomersByCity
mylist=custDAO.findCustomersByCity("Hyd");
mylist.forEach(System.out::println);

//5. getCustomersByEmailAndPhone
mylist=custDAO.getCustomersByEmailAndPhone("hello@jlc", 12345);
mylist.forEach(System.out::println);
```



```
//6. getCustomersByEmailAndPhone
mylist=custDAO.getCustomersByEmailOrPhone("hello@jlc", 12345);
mylist.forEach(System.out::println);

//7. getCustomersByCnameLike
mylist=custDAO.getCustomersByCnameLike("sr%");
mylist.forEach(System.out::println);

//8. getCustomersByCnameStartsWith
mylist=custDAO.getCustomersByCnameStartsWith("sd");
mylist.forEach(System.out::println);

//9. getCustomersByEmailEndsWith
mylist=custDAO.getCustomersByEmailEndsWith("@cc");
mylist.forEach(System.out::println);

//10. getCustomersByEmailContains
mylist=custDAO.getCustomersByCnameContains("jlc");
mylist.forEach(System.out::println);

//11. getCustomersByCityIn
mylist=custDAO.getCustomersByCityIn("Hyd","Delhi");
mylist.forEach(System.out::println);

//12. getCustomersByCityNotIn
mylist=custDAO.getCustomersByCityNotIn("Hyd","Delhi");
mylist.forEach(System.out::println);

//13. countCustomersByCity
long count=custDAO.countCustomersByCity("Hyd");
System.out.println(count);

//14. countCustomersByCityIn
count=custDAO.countCustomersByCityIn("Hyd","Delhi");
System.out.println(count);

//15. getCustomersByCityNotIn
count=custDAO.countCustomersByCityNotIn("Blore");
System.out.println(count);

//16. countCustomersByEmailNull
count=custDAO.countCustomersByEmailNull();
System.out.println(count);

//17. countCustomersByEmailNotNull
count=custDAO.countCustomersByEmailNotNull();
System.out.println(count);
```



```
//18. getCustomersByEmailNull
mylist=custDAO.getCustomersByEmailNull();
mylist.forEach(System.out::println);

//19. getCustomersByEmailNotNull
mylist=custDAO.getCustomersByEmailNotNull();
mylist.forEach(System.out::println);

//20. getCustomersByBalance
mylist=custDAO.getCustomersByBalance(15000);
mylist.forEach(System.out::println);

//21. getCustomersByBalanceIsBetween
mylist=custDAO.getCustomersByBalanceIsBetween(10000,20000);
mylist.forEach(System.out::println);

//22. getCustomersByBalanceIsGreaterThan
mylist=custDAO.getCustomersByBalanceIsGreaterThan(20000);
mylist.forEach(System.out::println);

//23. getCustomersByBalanceLessThan
mylist=custDAO.getCustomersByBalanceLessThan(10000);
mylist.forEach(System.out::println);

//24. getCustomersByBalanceGreaterThanOrEqualTo
mylist=custDAO.getCustomersByBalanceGreaterThanOrEqualTo(20000);
mylist.forEach(System.out::println);

//25. getCustomersByBalanceLessThanOrEqualTo
mylist=custDAO.getCustomersByBalanceLessThanOrEqualTo(10000);
mylist.forEach(System.out::println);

System.out.println("Done!!!");

}

public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}
}
```

2) Customer.java

```
package com.jlcindia.springboot;

import javax.persistence.*;
/*
```



* **@Author : Srinivas Dande**

* **@Company: Java Learning Center**

* */

@Entity

@Table(name="mycustomers")

public class Customer {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

@Column(name="cid")

private int cid;

@Column(name="cname")

private String cname;

@Column(name="email")

private String email;

@Column(name="phone")

private int phone;

@Column(name="city")

private String city;

@Column(name="balance")

private double balance;

public Customer() {}

public Customer(String cname, String email, int phone, String city,double balance) {

this.cname = cname;

this.email = email;

this.phone = phone;

this.city = city;

this.balance=balance;

}

public Customer(int cid, String cname, String email, int phone, String city,double balance) {

this.cid = cid;

this.cname = cname;

this.email = email;

this.phone = phone;

this.city = city;

this.balance=balance;

}

//Setters and Getters

//toString()

}



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO extends JpaRepository<Customer, Integer> {

    List<Customer> getCustomersByCname(String cname);
    List<Customer> getCustomersByEmail(String email);
    List<Customer> readCustomersByPhone(int phone);
    List<Customer> findCustomersByCity(String city);

    List<Customer> getCustomersByEmailAndPhone(String email, int phone);
    List<Customer> getCustomersByEmailOrPhone(String email, int phone);

    List<Customer> getCustomersByCnameLike(String cname);
    List<Customer> getCustomersByCnameStartsWith(String cname);
    List<Customer> getCustomersByEmailEndsWith(String email);
    List<Customer> getCustomersByCnameContains(String email);

    List<Customer> getCustomersByCityIn(String... cities);
    List<Customer> getCustomersByCityNotIn(String... cities);

    long countCustomersByCity(String city);
    long countCustomersByCityIn(String... cities);
    long countCustomersByCityNotIn(String... cities);
    long countCustomersByCnameLike(String cname);

    long countCustomersByEmailNull();
    long countCustomersByEmailNotNull();

    List<Customer> getCustomersByEmailNull();
    List<Customer> getCustomersByEmailNotNull();

    List<Customer> getCustomersByBalance(double bal);
    List<Customer> getCustomersByBalanceIsBetween(double bal1, double bal2);
    List<Customer> getCustomersByBalanceIsGreaterThan(double bal);

    List<Customer> getCustomersByBalanceLessThan(double bal);
    List<Customer> getCustomersByBalanceGreaterThanEqual(double bal);
    List<Customer> getCustomersByBalanceLessThanEqual(double bal);
}
```



6) mycustomers table

```
create table mycustomers(  
  cid int primary key,  
  cname char(10),  
  email char(10),  
  phone long,  
  city char(10) ,  
  balance double );
```

demo4-data-jpa: Files required

1. MyBootApplication.java	Updated in Demo4
2. Customer.java	Same as Demo3
3. CustomerDAO.java	Updated in Demo4
4. application.properties	Same as Demo3
5. pom.xml	Same as Demo3
6. mycustomers table	Same as Demo3

1) MyBootApplication.java

```
package com.jlcindia.springboot;  
  
import java.util.List;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.boot.CommandLineRunner;  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
@SpringBootApplication  
public class MyBootApplication implements CommandLineRunner {  
  
    @Autowired  
    CustomerDAO custDAO;  
  
    @Override  
    public void run(String... args) throws Exception {  
  
        List<Customer> mylist=null;
```



```
//1. getCustomersNyEmailAndPhone1
mylist=custDAO.getCustomersNyEmailAndPhone1("hello@jlc",12345);
mylist.forEach(System.out::println);

//2. getCustomersNyEmailAndPhone2
mylist=custDAO.getCustomersNyEmailAndPhone2("hello@jlc",12345);
mylist.forEach(System.out::println);

//3. getCustomersNyEmailOrPhone1
mylist=custDAO.getCustomersNyEmailOrPhone1("hello@jlc",12345);
mylist.forEach(System.out::println);

//4. getCustomersNyEmailOrPhone2
mylist=custDAO.getCustomersNyEmailOrPhone2("hello@jlc",12345);
mylist.forEach(System.out::println);

//5. findCustomersByCity1
mylist=custDAO.findCustomersByCity1("Blore");
mylist.forEach(System.out::println);

//6. findCustomersByCity2
mylist=custDAO.findCustomersByCity2("Blore");
mylist.forEach(System.out::println);

//7. findAllCustomers1
mylist=custDAO.findAllCustomers1();
mylist.forEach(System.out::println);

//8. findAllCustomers2
mylist=custDAO.findAllCustomers2();
mylist.forEach(System.out::println);

//9. findCustomersByPhone1
mylist=custDAO.findCustomersByPhone1(12345);
mylist.forEach(System.out::println);

//10. findCustomersByPhone2
mylist=custDAO.findCustomersByPhone2(12345);
mylist.forEach(System.out::println);

System.out.println("Done!!!");
}
public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}
}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO extends JpaRepository<Customer, Integer> {

    @Query(value = "from Customer cust where email=?1 and phone=?2")
    List<Customer> getCustomersNyEmailAndPhone1(String email, int phone);

    @Query(value = "select * from mycustomers cust where email=?1 and phone=?2",
    nativeQuery = true)
    List<Customer> getCustomersNyEmailAndPhone2(String email, int phone);

    @Query(value = "from Customer cust where email=?1 or phone=?2")
    List<Customer> getCustomersNyEmailOrPhone1(String email, int phone);

    @Query(value = "select * from mycustomers cust where email=?1 or phone=?2",
    nativeQuery = true)
    List<Customer> getCustomersNyEmailOrPhone2(String email, int phone);

    @Query(value = "from Customer cust where city=?1 order by cname desc")
    List<Customer> findCustomersByCity1(String city);

    @Query(value = "select * from myustomers cust where city=?1 order by cname desc",
    nativeQuery = true)
    List<Customer> findCustomersByCity2(String city);

    @Query(value = "from Customer cust order by cname asc")
    List<Customer> findAllCustomers1();

    @Query(value = "select * from myustomers cust order by cname asc", nativeQuery = true)
    List<Customer> findAllCustomers2();

    @Query(value = "from Customer cust where cust.phone = :myphone")
    List<Customer> findCustomersByPhone1(@Param("myphone") long phone);

    @Query(value = "select * from customers where cust.phone = :myphone", nativeQuery
    = true)
    List<Customer> findCustomersByPhone2(@Param("myphone") long phone);
}
```